

CSE 4810 : Algorithm Engineering Lab

Lab - 0

MD ATIQUR RAHMAN

Group - CSE 1B

May 22, 2025

Task 1

Given an array `arr[]` of integers, the task is to find the Next Greater Element for each element of the array in order of their appearance in the array. You need to give a solution with $O(n)$ time and $O(n)$ space complexity.

Note: The Next Greater Element for an element x is the first greater element on the right side of x in the array. Elements for which no greater element exist, consider the next greater element as -1 .

Input: `arr[] = [1, 3, 2, 4]`

Output: `[3, 4, 4, -1]`

Input: `arr[] = [6, 8, 0, 1, 3]`

Output: `[8, -1, 1, 3, -1]`

Task 2

Implement Stack using two Queues (`q1` and `q2`) following the below algorithm.

- Steps for `push(x)`:
 - Enqueue x to `q2`.
 - One by one dequeue everything from `q1` and enqueue to `q2`.
 - Swap the queues of `q1` and `q2`.
- Steps for `pop()`:
 - Dequeue an item from `q1` and return it.

Task 3

Given an array `arr[]` of integers and an integer k , your task is to find the maximum value for each contiguous subarray of size k . The output should be an array of maximum values corresponding to each contiguous subarray. Time complexity is $O(n)$ and Space Complexity constraint is $O(k)$.

Input: `arr[] = [1, 2, 3, 1, 4, 5, 2, 3, 6]`, $k = 3$

Output: `[3, 3, 4, 5, 5, 5, 6]`

Explanation:

1st contiguous subarray = `[1 2 3]` $\text{max} = 3$

2nd contiguous subarray = `[2 3 1]` $\text{max} = 3$

3rd contiguous subarray = [3 1 4] max = 4
 4th contiguous subarray = [1 4 5] max = 5
 5th contiguous subarray = [4 5 2] max = 5
 6th contiguous subarray = [5 2 3] max = 5
 7th contiguous subarray = [2 3 6] max = 6

Input: arr[] = [5, 1, 3, 4, 2, 6], k = 1

Output: [5, 1, 3, 4, 2, 6]

Explanation: When k = 1, each element in the array is its own subarray, so the output is simply the same array.

Input: arr[] = [1, 3, 2, 1, 7, 3], k = 3

Output: [3, 3, 7, 7]

Helping Code

Implementation of Stack

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        """Push an item onto the stack."""
        self.items.append(item)

    def pop(self):
        """Pop an item off the stack. Returns None if the stack is empty."""
        if not self.empty():
            return self.items.pop()
        else:
            return None

    def empty(self):
        """Check if the stack is empty."""
        return len(self.items) == 0

    def size(self):
        """Return the number of items in the stack."""
        return len(self.items)

# Example usage:
if __name__ == "__main__":
    s = Stack()
    s.push(10)
    s.push(20)
    print("Size:", s.size()) # Output: 2
    print("Pop:", s.pop())   # Output: 20
    print("Empty:", s.empty()) # Output: False
    s.pop()
    print("Empty after popping all:", s.empty()) # Output: True
```

Implementation of Deque

If you can access collections module. In Python 3.12.6 you can.

```
from collections import deque

# Create a deque
dq = deque()

# Add to the right (like queue)
dq.append(1)
dq.append(2)

# Add to the left
dq.appendleft(0)

# Remove from the right
dq.pop() # Removes 2

# Remove from the left
dq.popleft() # Removes 0

# Check if empty
print(len(dq) == 0)

# View the deque
print(dq) # deque([1])
```

If you can not then use the following implementation. Remember the following will cost you $O(n)$ for some operation while the above implementation will only cost you $O(1)$.

```
class Deque:
    def __init__(self):
        self.items = []

    def append(self, item):
        """Add to the right end."""
        self.items.append(item)

    def appendleft(self, item):
        """Add to the left end."""
        self.items.insert(0, item)

    def pop(self):
        """Remove and return item from right end."""
        if not self.empty():
            return self.items.pop()
        return None

    def popleft(self):
        """Remove and return item from left end."""
        if not self.empty():
            return self.items.pop(0)
        return None
```

```

def empty(self):
    """Check if deque is empty."""
    return len(self.items) == 0

def size(self):
    """Return the number of items."""
    return len(self.items)

def __str__(self):
    return str(self.items)

# Example usage
dq = Deque()
dq.append(10)
dq.appendleft(5)
dq.append(15)
print(dq)           # Output: [5, 10, 15]
print(dq.popleft()) # Output: 5
print(dq.pop())     # Output: 15
print(dq)           # Output: [10]

```